

Chapter 10 Maps and Dictionaries

Maps and Dictionaries

◆ Prompt 1: What is a Map (or Dictionary) in Python?

✓ Answer:

A **map** (or dictionary in Python) is an **abstract data type** that stores **key-value pairs**, where:

- Each **key** is unique
- Each **key** maps to a **value**
- Values do **not have to be unique**

Python implements this as the `dict` class.

Example:

```
currency = {  
    "Turkey": "Lira",  
    "Spain": "Euro",  
    "Greece": "Euro",  
    "China": "Yuan",  
    "United States": "Dollar",  
    "India": "Rupee"  
}
```

◆ Prompt 2: What is the difference between a key and a value?

✓ Answer:

- **Key:** A unique identifier used to access a value.
- **Value:** The data associated with a key.

Example:

```
currency["Spain"]  # Key: "Spain"  
                  # Value: "Euro"
```

- Keys must be **immutable** (numbers, strings, tuples, etc.)
- Values can be **any Python object**

◆ Prompt 3: What is the difference between an array and a map?

Feature	Array	Map (Dictionary)
Indexing	Numeric, consecutive	Any immutable type (string, number, tuple)
Uniqueness	Indices unique by position	Keys must be unique
Lookup	O(1) by index	O(1) average by key (hash-based)
Values	Any type	Any type
Example	arr[0] = 5	currency["Spain"] = "Euro"

◆ Prompt 4: How do you access or update a value in a Python dictionary?

✓ Answer:

Access a value:

```
currency["Greece"] # Output: "Euro"
```

Update a value:

```
currency["Greece"] = "Drachma"
```

Add a new key-value pair:

```
currency["Japan"] = "Yen"
```

◆ Prompt 5: Why are maps/dictionaries useful?

✓ Answer:

Maps/dictionaries are useful because:

1. They allow **fast lookup** of data using a key.
2. They allow **associating arbitrary values** with unique identifiers.
3. Keys can be **strings, numbers, or tuples**, not just numeric indices.
4. Values can be any Python object, including other dictionaries.

◆ Prompt 6: Give real-world applications of maps

✓ Answer:

Application	Description
Student Records	Student ID → Name, address, grades
DNS	Host name → IP address
Social Media	Username → User information
Computer Graphics	Color name → RGB values
Python Namespaces	Variable name → Object/value

◆ Prompt 9: Write a simple Python example using a dictionary

✓ Answer:

```
# Map of countries to currency
currency = {
    "Turkey": "Lira",
    "Spain": "Euro",
    "Greece": "Euro",
    "China": "Yuan",
    "United States": "Dollar",
    "India": "Rupee"
```

```
}

# Access a value
print(currency["India"]) # Output: Rupee

# Update a value
currency["Greece"] = "Drachma"

# Add a new key-value pair
currency["Japan"] = "Yen"

# Check if key exists
if "China" in currency:
    print("China uses", currency["China"])
```

The Map ADT (Abstract Data Type)

◆ Prompt 1: What is a Map ADT?

✓ Answer:

A **map** (also called a dictionary or associative array) is an **abstract data type** that stores **key-value pairs**:

- **Keys** are unique
- **Values** can be any type

A map supports operations to:

- Query values using keys
- Add or modify key-value pairs
- Delete pairs
- Iterate over keys

In Python, the `dict` class implements the map ADT.

◆ Prompt 2: What are the five core behaviors of a map?

✓ Answer:

$M[k]$

- Return the value associated with key k .
- Raises `KeyError` if k does not exist.

$M[k] = v$

- Insert or update key k with value v .

`del M[k]`

- Remove the key-value pair for k .
- Raises `KeyError` if k does not exist.

`len(M)`

- Return the number of key-value pairs.

iter(M)

- Iterate over the keys of the map.

◆ Prompt 3: What additional convenient methods do maps provide?

✓ Answer:

- **k in M** → Check if key exists
- **M.get(k, d=None)** → Return $M[k]$ if exists, else d
- **M.setdefault(k, d)** → Return $M[k]$ if exists, else set $M[k]=d$
- **M.pop(k, d=None)** → Remove key and return value; return d if missing
- **M.popitem()** → Remove an arbitrary key-value pair
- **M.clear()** → Remove all items
- **M.keys()** → View of keys
- **M.values()** → View of values
- **M.items()** → View of key-value pairs
- **M.update(M2)** → Add all key-value pairs from M2
- **M == M2 / M != M2** → Compare key-value associations

◆ Prompt 4: Show step-by-step example of map operations

✓ Answer:

Initial empty map: $M = \{\}$

Operation	Return Value	Map Contents
<code>len(M)</code>	0	<code>{}</code>
<code>M['K'] = 2</code>	—	<code>{'K':2}</code>
<code>M['B'] = 4</code>	—	<code>{'K':2, 'B':4}</code>
<code>M['U'] = 2</code>	—	<code>{'K':2, 'B':4, 'U':2}</code>
<code>M['V'] = 8</code>	—	<code>{'K':2, 'B':4, 'U':2, 'V':8}</code>
<code>M['K'] = 9</code>	—	<code>{'K':9, 'B':4, 'U':2, 'V':8}</code>
<code>M['B']</code>	4	<code>{'K':9, 'B':4, 'U':2, 'V':8}</code>
<code>M['X']</code>	KeyError	<code>{'K':9, 'B':4, 'U':2, 'V':8}</code>
<code>M.get('F')</code>	None	<code>{'K':9, 'B':4, 'U':2, 'V':8}</code>
<code>M.get('F', 5)</code>	5	<code>{'K':9, 'B':4, 'U':2, 'V':8}</code>
<code>M.get('K', 5)</code>	9	<code>{'K':9, 'B':4, 'U':2, 'V':8}</code>
<code>len(M)</code>	4	<code>{'K':9, 'B':4, 'U':2, 'V':8}</code>
<code>del M['V']</code>	—	<code>{'K':9, 'B':4, 'U':2}</code>

M.pop('K')	9	{'B':4,'U':2}
M.keys()	['B','U']	{'B':4,'U':2}
M.values()	[4,2]	{'B':4,'U':2}
M.items()	[('B',4),('U',2)]	{'B':4,'U':2}
M.setdefault('B',1)	4	{'B':4,'U':2}
M.setdefault('A',1)	1	{'A':1,'B':4,'U':2}
M.popitem()	('B',4)	{'A':1,'U':2}

Python's MutableMapping Abstract Base Class

◆ Prompt 1: What is an Abstract Base Class (ABC) in Python?

✓ Answer:

An **abstract base class (ABC)** is a class that:

- Cannot be instantiated directly
- Defines a **set of required methods**
- Forces subclasses to implement those methods

Abstract base classes:

- Define **what a class should do**, not **how**
- Are commonly used to enforce **consistent interfaces**

Python provides ABCs through the `collections.abc` module.

◆ Prompt 2: What are Mapping and MutableMapping?

✓ Answer:

Python provides two important abstract base classes for map-like data structures:

◆ Mapping

- Represents **read-only maps**
- Includes **non-mutating methods**
- Example: lookup, iteration, membership tests

◆ **MutableMapping**

- Extends `Mapping`
- Adds **mutating operations**
- Example: insert, delete, update

★ Python's built-in `dict` is a **concrete implementation** of `MutableMapping`.

◆ **Prompt 3: How does `MutableMapping` relate to the Map ADT?**

✓ **Answer:**

The **Map ADT** defined earlier is **conceptually equivalent** to Python's `MutableMapping`.

Map ADT	MutableMapping
Key-value pairs	Key-value pairs
Lookup	<code>__getitem__</code>
Insert/update	<code>__setitem__</code>
Delete	<code>__delitem__</code>
Size	<code>__len__</code>
Iteration	<code>__iter__</code>

★ If a class implements **only five core methods**, `MutableMapping` supplies **all remaining map behaviors automatically**.

◆ **Prompt 4: What are the five abstract methods required by `MutableMapping`?**

✓ **Answer:**

Any class inheriting from `MutableMapping` **must implement these five methods**:

1. `__getitem__(self, k)`
2. `__setitem__(self, k, v)`
3. `__delitem__(self, k)`
4. `__len__(self)`
5. `__iter__(self)`

These are the **core behaviors** of a map.

◆ Prompt 5: What methods are automatically provided by `MutableMapping`?

✓ Answer:

Once the five core methods are implemented, `MutableMapping` provides:

- `__contains__` $\rightarrow k \text{ in } M$
- `get`
- `setdefault`
- `pop`
- `popitem`
- `clear`
- `keys`
- `values`
- `items`
- `update`
- Equality tests (`==`, `!=`)

✓ You **do not** need to write these yourself.

◆ Prompt 6: How is the `__contains__` method derived?

✓ Answer:

The `k in M` operation is implemented using `__getitem__`.

Example implementation:

```
def __contains__(self, k):  
    try:  
        self[k]          # calls __getitem__  
        return True  
    except KeyError:  
        return False
```

Explanation:

- Try accessing the key

- If it exists → return `True`
- If `KeyError` occurs → return `False`

◆ Prompt 7: How is `setdefault` derived?

✓ Answer:

```
setdefault(k, d):
```

- Returns the value if key exists
- Otherwise inserts default value and returns it

Example implementation:

```
def setdefault(self, k, d):
    try:
        return self[k]          # __getitem__
    except KeyError:
        self[k] = d             # __setitem__
    return d
```

◆ Prompt 8: Example of a user-defined map using `MutableMapping`

✓ Answer:

```
from collections.abc import MutableMapping

class SimpleMap(MutableMapping):
    def __init__(self):
        self._data = []

    def __getitem__(self, k):
        for key, value in self._data:
            if key == k:
                return value
        raise KeyError(k)
```

```

def __setitem__(self, k, v):
    for i, (key, _) in enumerate(self._data):
        if key == k:
            self._data[i] = (k, v)
            return
    self._data.append((k, v))

def __delitem__(self, k):
    for i, (key, _) in enumerate(self._data):
        if key == k:
            self._data.pop(i)
            return
    raise KeyError(k)

def __len__(self):
    return len(self._data)

def __iter__(self):
    for key, _ in self._data:
        yield key

```

✦ This class automatically supports:

`M.get()`

`M.setdefault()`

`k in M`

`M.pop()`

`M.keys()`

Simple Unsorted Map Implementation

(UnsortedTableMap)

◆ Prompt 1: What is an UnsortedTableMap?

✓ Answer:

An **UnsortedTableMap** is a simple implementation of the **Map ADT** that stores key–value pairs in **no particular order** inside a **Python list**.

- Keys are **not sorted**
- Elements are stored sequentially
- Each element is an **Item (key, value)** object

✦ It is mainly used for **learning and demonstration**, not for performance.

◆ Prompt 2: What data structure does UnsortedTableMap use internally?

✓ Answer:

It uses a **Python list** to store map entries.

```
self._table = []
```

Each entry in the list is an **Item object** containing:

- `item.key`
- `item.value`

◆ Prompt 3: What is the role of MapBase?

✓ Answer:

MapBase is a **parent class** that:

- Inherits from `MutableMapping`
- Defines a nested `Item` class

```
class Item:
```

```
    __slots__ = '_key', '_value'
```

✦ This allows all map implementations to reuse the same `(key, value)` structure.

◆ Prompt 4: How does `__getitem__` work?

✓ **Answer:**

`__getitem__(k)` searches the list **linearly** for a matching key.

Code logic:

```
for item in self._table:
    if k == item.key:
        return item.value
raise KeyError
```

Explanation:

- Scan items one by one
- If key is found → return value
- If not found → raise `KeyError`

🕒 **Time Complexity:**

- Best case: **$O(1)$**
- Worst case: **$O(n)$**

◆ Prompt 5: How does `__setitem__` work?

✓ **Answer:**

`__setitem__(k, v):`

- Updates value if key exists
- Adds a new item if key does not exist

Code logic:

```
for item in self._table:
    if k == item.key:
        item.value = v
return
```

```
self._table.append(Item(k, v))
```

✦ No sorting or hashing is used.

🕒 **Time Complexity:**

- Worst case: $O(n)$

◆ Prompt 6: How does `__delitem__` work?

✓ **Answer:**

Deletes an item by:

- Searching for the key
- Removing it using `pop`

Code logic:

```
for j in range(len(self._table)):
    if k == self._table[j].key:
        self._table.pop(j)
        return
raise KeyError
```

🕒 **Time Complexity:**

- Worst case: $O(n)$

◆ Prompt 7: How does `__len__` work?

✓ **Answer:**

Returns the number of items stored in the map.

```
return len(self._table)
```

🕒 **Time Complexity:**

- $O(1)$

◆ Prompt 8: How does `__iter__` work?

✓ Answer:

Iterates over **keys only**, not values.

```
for item in self._table:
    yield item.key
```

✦ Required by the Map ADT.

◆ Prompt 9: Why is this implementation inefficient?

✓ Answer:

Because **every operation scans the entire list**.

Operation	Time Complexity
getitem	$O(n)$
setitem	$O(n)$
delitem	$O(n)$
len	$O(1)$
iter	$O(n)$

✦ This makes it unsuitable for large datasets.

◆ Prompt 10: Complete code (clean version)

✓ Answer:

```
from collections.abc import MutableMapping
```

```
class MapBase(MutableMapping):
```

```
    class Item:
```

```
        __slots__ = '_key', '_value'
```

```
        def __init__(self, k, v):
```

```
            self._key = k
```

```
            self._value = v
```

```
class UnsortedTableMap(MapBase):

    def __init__(self):
        self._table = []

    def __getitem__(self, k):
        for item in self._table:
            if k == item._key:
                return item._value
        raise KeyError(k)

    def __setitem__(self, k, v):
        for item in self._table:
            if k == item._key:
                item._value = v
                return
        self._table.append(self.Item(k, v))

    def __delitem__(self, k):
        for i in range(len(self._table)):
            if k == self._table[i]._key:
                self._table.pop(i)
                return
        raise KeyError(k)

    def __len__(self):
        return len(self._table)
```



```
def __iter__(self):  
    for item in self._table:  
        yield item._key
```